



hashcat

advanced
password
recovery

hashcat Forums Wiki Tools Events

Recent changes Admin Log In Sitemap

Overview

Table of Contents

- Overview
- Pre-Installation
 - Why does a window pop up and close immediately?
 - I am a complete noob, what can I do for getting started?
 - Why are there different versions of *hashcat?
 - Why are there so many binaries, which one should I use?
 - How do I verify the PGP signatures?
 - Linux
 - Windows
 - Is there a hashcat GUI?
- Installation
 - How do I install hashcat?
 - How does one install the correct driver for the GPU(s)?
 - GPU device not found, why?
 - I may have the wrong driver installed, what should I do?
 - What does the cuModuleLoad() 209 error mean?
 - What does the cuModuleLoad() 301 error mean?
 - What does the clGetPlatformIDs() -64 error mean?
 - What does the clGetPlatformIDs() -1001 error mean?
 - What does the clGetDeviceIDs() -1 Error error message mean?
 - What does the clBuildProgram() -11 error mean?
 - What does the clCreateBuffer() -61 error mean?
 - What does the clEnqueueCopyBuffer() -30 error mean?
 - What does the cuStreamSynchronize() 702 error mean?
 - What does the error: FanSpeed_Get(): -5 mean?
 - What does the ADL_Overdrive5_Temperature_Get(): -1 error mean?
 - I got this error message: undefined symbol: nvmlInit_v2 Error, what does it mean?
 - What does the clEnqueueNDRangeKernel() : -4 : CL_MEM_OBJECT_ALLOCATION_FAILURE error mean?
 - How can I use old hashcat/oclHashcat versions?
- Post-Installation
 - I see only the usage, it seems to be a syntax error. What should I do?
 - hashcat unpacks to 1GB or more, do I really need all the files?
- Basic cracking knowledge
 - What does "Status: Exhausted" mean?
 - What is a hashcat mask file?
 - What exactly is a weak hash?
 - What is a potfile?
 - How can I identify the hash type?
 - Hashcat reports "Status: Cracked", but did not print the hash value, and the outfile is empty. What happened?
 - How can I show previously cracked passwords, and output them in a specific format (e.g. email:password)?
 - What is a keyspaces?
 - How to deal with special non-latin characters (chinese, arabic, etc...) ?
- Attack-Modes
 - Why should I use a mask attack? I just want to "brute" these hashes!
 - I do not know the password length, how can I increment the length of the password candidates?
 - I want to optimize my Brute-Force attack by ordering letters by frequency in a custom charset. How to do it? Does it make sense?
 - I want to use rules, but there are three different parameters. When do I use -r, -j and -k?
 - How does one use several rules at once?
 - OK, there is a hybrid attack for append mask and prepend mask, but what if I want to use both at the same time?
 - When I use --increment in hybrid attack how does that work?
 - How to use multiple dictionaries?
 - Why are there 5 different toggle rules in the rules/ folder?
 - When I run an attack with -a 3 and I do not specifying a mask, I see it working but what is it doing?
 - How does one use the new prince attack mode with hashcat legacy?
 - I have a half-known password. I know the first 4 letters, can hashcat get the rest of the password?
 - Why are there two different hash-modes for Vbulletin?
- Performance
 - How much faster is cracking on Linux compared to a Windows operating system?
 - How can I perform a benchmark?
 - My desktop lags too much, anything I can do to avoid it?
 - Is the 64 bit version faster than the 32 bit version?
 - What is it that you call "GPU power"?
 - How to create more work for full speed?
 - Is piping a wordlist slower than reading from file?
 - Why is my attack so slow?
 - Why does hashcat says it has only 2% GPU utilization?
 - How is it possible that hashcat does not utilize all GPUs?
 - Why does hashcat sometimes get very slow at the end of an attack?
 - Why is hashcat taking so long to report progress?
- Special features
 - Can I restore a hashcat session?
 - Can I restart hashcat on a different PC or is it possible to add a new GPU to my system?
 - How can I distribute the work on different computers / nodes?
 - I read somewhere to use VCL for distributed cracking, is this still a thing?
 - Can hashcat send an email once a hash has been found?
 - Why is my script unable to communicate with the hashcat / hashcat legacy prompt?
 - I got a hash cracked on a different computer, can I tell hashcat about that while it is running?
 - How do I extract the hashes from Office (Word, Excel, etc.) documents?
 - How do I extract the hashes from PDF documents?
 - How do I extract the hashes from TrueCrypt volumes?
 - How do I extract the hashes from VeraCrypt volumes?

Table of Contents

- Overview
- Pre-Installation
 - Why does a window pop up and close immediately?
 - I am a complete noob, what can I do for getting started?
 - Why are there different versions of *hashcat?
 - Why are there so many binaries, which one should I use?
 - How do I verify the PGP signatures?
 - Linux
 - Windows
 - Is there a hashcat GUI?
- Installation
 - How do I install hashcat?
 - How does one install the correct driver for the GPU(s)?
 - GPU device not found, why?
 - I may have the wrong driver installed, what should I do?
 - What does the cuModuleLoad() 209 error mean?
 - What does the cuModuleLoad() 301 error mean?
 - What does the clGetPlatformIDs() -64 error mean?
 - What does the clGetPlatformIDs() -1001 error mean?
 - What does the clGetDeviceIDs() -1 Error error message mean?
 - What does the clBuildProgram() -11 error mean?
 - What does the clCreateBuffer() -61 error mean?
 - What does the clEnqueueCopyBuffer() -30 error mean?
 - What does the cuStreamSynchronize() 702 error mean?
 - What does the error: FanSpeed_Get(): -5 mean?
 - What does the ADL_Overdrive5_Temperature_Get(): -1 error mean?
 - I got this error message: undefined symbol: nvmlInit_v2 Error, what does it mean?
 - What does the clEnqueueNDRangeKernel() : -4 : CL_MEM_OBJECT_ALLOCATION_FAILURE error mean?
 - How can I use old hashcat/oclHashcat versions?
- Post-Installation
 - I see only the usage, it seems to be a syntax error. What should I do?
 - hashcat unpacks to 1GB or more, do I really need all the files?
- Basic cracking knowledge
 - What does "Status: Exhausted" mean?
 - What is a hashcat mask file?
 - What exactly is a weak hash?
 - What is a potfile?
 - How can I identify the hash type?
 - Hashcat reports "Status: Cracked", but did not print the hash value, and the outfile is empty. What happened?
 - How can I show previously cracked passwords, and output them in a specific format (e.g. email:password)?
 - What is a keyspaces?
 - How to deal with special non-latin characters (chinese, arabic, etc...) ?
- Attack-Modes
 - Why should I use a mask attack? I just want to "brute" these hashes!
 - I do not know the password length, how can I increment the length of the password candidates?
 - I want to optimize my Brute-Force attack by ordering letters by frequency in a custom charset. How to do it? Does it make sense?
 - I want to use rules, but there are three different parameters. When do I use -r, -j and -k?
 - How does one use several rules at once?
 - OK, there is a hybrid attack for append mask and prepend mask, but what if I want to use both at the same time?
 - When I use --increment in hybrid attack how does that work?
 - How to use multiple dictionaries?
 - Why are there 5 different toggle rules in the rules/ folder?
 - When I run an attack with -a 3 and I do not specifying a mask, I see it working but what is it doing?
 - How does one use the new prince attack mode with hashcat legacy?
 - I have a half-known password. I know the first 4 letters, can hashcat get the rest of the password?
 - Why are there two different hash-modes for Vbulletin?
- Performance
 - How much faster is cracking on Linux compared to a Windows operating system?

- How can I crack passwords from htpasswd?
- How can I crack SL3?
- How can I crack multiple WPA handshakes at once?
- What means "rejected" in the status view?
- What is the maximum supported password length?
- What is the maximum supported salt length?
- What is the maximum supported password length for optimized kernels?
- What is the maximum supported salt length for optimized kernels?
- I do not want hashcat to reject words from my wordlist if they are too long, can it truncate them instead?
- How can I perform a combinator attack with three wordlists (triple combinator, 3-way combinator)?
- Can I load multiple hashlists at once?
- I want to skip password candidates that have repeating characters. Is this possible?
- LM consists of two hashes but are split for use with hashcat. How to get them back into one piece?
- Hardware
 - How can I make sure that my GPU is supported?
 - What is the best GPU for hashcat for "total speed", "speed by watt" and "speed by price"?
 - Is there a general benchmark table for all GPU?
 - Does the PCI-Express speed have any influence on cracking speed?
 - Should I attach Crossfire (AMD) / SLI (Nvidia) bridges?
 - My GPU overheats sometimes. Can hashcat automatically pause in case of overheating or with a script?
 - Why does the temperature reported on the status screen not match the temperature in Afterburner?
 - How many GPU can hashcat handle?
 - I was doing bitcoin and litecoin mining, can I use my FPGA or ASIC for hashcat?
 - Can I use Intel GPUs, they support OpenCL?
 - Why does hashcat show the wrong GPU memory when I start it?
- Common errors
 - I created a hash with "echo test | md5sum" but hashcat fails to crack it
 - Line-length exception, wait what?
 - When I use --remove with hashcat legacy the hash sometimes remains even if it was cracked
- Compatibility
 - Why does hashcat not work with my Kali operating system?
 - Can I use JtR rules with hashcat?
 - Which wordlist are recommended for WPA cracking?
- Organizational
 - I want to request some new algorithms or features, how can I accomplish this?
 - Where can I download the source code?
 - What is Team Hashcat?
- New Questions, TBD
 - Is it worth renting Amazon GPUs rather than buying GPUs?

- How can I perform a benchmark?
- My desktop lags too much, anything I can do to avoid it?
- Is the 64 bit version faster than the 32 bit version?
- What is it that you call "GPU power"?
- How to create more work for full speed?
- Is piping a wordlist slower than reading from file?
- Why is my attack so slow?
- Why does hashcat says it has only 2% GPU utilization?
- How is it possible that hashcat does not utilize all GPUs?
- Why does hashcat sometimes get very slow at the end of an attack?
- Why is hashcat taking so long to report progress?
- Special features
 - Can I restore a hashcat session?
 - Can I restart hashcat on a different PC or is it possible to add a new GPU to my system?
 - How can I distribute the work on different computers / nodes?
 - I read somewhere to use VCL for distributed cracking, is this still a thing?
 - Can hashcat send an email once a hash has been found?
 - Why is my script unable to communicate with the hashcat / hashcat legacy prompt?
 - I got a hash cracked on a different computer, can I tell hashcat about that while it is running?
 - How do I extract the hashes from Office (Word, Excel, etc.) documents?
 - How do I extract the hashes from PDF documents?
 - How do I extract the hashes from TrueCrypt volumes?
 - How do I extract the hashes from VeraCrypt volumes?
 - How can I crack passwords from htpasswd?
 - How can I crack SL3?
 - How can I crack multiple WPA handshakes at once?
 - What means "rejected" in the status view?
 - What is the maximum supported password length?
 - What is the maximum supported salt length?
 - What is the maximum supported password length for optimized kernels?
 - What is the maximum supported salt length for optimized kernels?
 - I do not want hashcat to reject words from my wordlist if they are too long, can it truncate them instead?
 - How can I perform a combinator attack with three wordlists (triple combinator, 3-way combinator)?
 - Can I load multiple hashlists at once?
 - I want to skip password candidates that have repeating characters. Is this possible?
 - LM consists of two hashes but are split for use with hashcat. How to get them back into one piece?
- Hardware
 - How can I make sure that my GPU is supported?
 - What is the best GPU for hashcat for "total speed", "speed by watt" and "speed by price"?
 - Is there a general benchmark table for all GPU?
 - Does the PCI-Express speed have any influence on cracking speed?
 - Should I attach Crossfire (AMD) / SLI (Nvidia) bridges?
 - My GPU overheats sometimes. Can hashcat automatically pause in case of overheating or with a script?
 - Why does the temperature reported on the status screen not match the temperature in Afterburner?
 - How many GPU can hashcat handle?
 - I was doing bitcoin and litecoin mining, can I use my FPGA or ASIC for hashcat?
 - Can I use Intel GPUs, they support OpenCL?
 - Why does hashcat show the wrong GPU memory when I start it?
- Common errors
 - I created a hash with "echo test | md5sum" but hashcat fails to crack it
 - Line-length exception, wait what?
 - When I use --remove with hashcat legacy the hash sometimes remains even if it was cracked
- Compatibility
 - Why does hashcat not work with my Kali operating system?
 - Can I use JtR rules with hashcat?
 - Which wordlist are recommended for WPA cracking?
- Organizational
 - I want to request some new algorithms or features, how can I accomplish this?
 - Where can I download the source code?
 - What is Team Hashcat?
- New Questions, TBD
 - Is it worth renting Amazon GPUs rather than buying GPUs?

Pre-Installation

Why does a window pop up and close immediately?

(short **URL**: <https://hashcat.net/faq/ubernoots>)

The reason for this is that the hashcat tools are command line tools only.

See the wiki page [Help for ubernoobs](#) to troubleshoot this problem.

I am a complete noob, what can I do for getting started?

The best way to get started with software from hashcat.net is to use the [wiki](#). Furthermore, you can use the [forum](#) to search for your specific questions (forum [search](#) function).

Please do not immediately start a new forum thread, first use the built-in search function and/or a web search engine to see if the question was already posted/answered

There are also some tutorials listed under [Howtos](#), [videos](#), [papers](#), [articles etc in the wild](#) to learn the very basics. Note these resources can be outdated

Why are there different versions of *hashcat?

- hashcat: A cracker for your GPU(s) and CPU(s) using OpenCL. It supports Nvidia, AMD and other OpenCL compatible devices
- hashcat legacy: A cracker for your CPU(s), it does not need, nor use your GPUs

Why are there so many binaries, which one should I use?

First, you need to know the details about your operating system:

- 32-bit operating system or 64-bit?
- Windows, Linux, or macOS?

Starting from this information, the selection of the correct binary goes like this:

- .bin are for Linux operating systems
- .exe are for Windows operating systems

For hashcat, the CPU usage should be very low for these binaries (if you do not utilize a OpenCL compatible CPU).

How do I verify the PGP signatures?

Linux

Start by downloading the signing key:

```
gpg --keyserver keys.gnupg.net --recv 8A16544F
```

Download the latest version of hashcat and its corresponding signature. For our example, we're going to use wget to download version 6.1.1:

```
wget https://hashcat.net/files/hashcat-6.1.1.7z
wget https://hashcat.net/files/hashcat-6.1.1.7z.asc
```

Verify the signature by running:

```
gpg --verify hashcat-6.1.1.7z.asc hashcat-6.1.1.7z
```

Your output will look like this:

```
gpg: Signature made Wed 29 Jul 2020 12:25:34 PM CEST
gpg:          using RSA key A70833229D040B4199CC00523C17DA8B8A16544F
gpg: Good signature from "Hashcat signing key <signing@hashcat.net>" [unknown]
gpg: WARNING: This key is not certified with a trusted signature!
gpg:          There is no indication that the signature belongs to the owner.
Primary key fingerprint: A708 3322 9D04 0B41 99CC 0052 3C17 DA8B 8A16 544F
```

Manually inspect the key fingerprint to assure that it matches what's on the website.

Windows

1. Download  GPG4Win. You want the top download, which will give you a graphical front-end named Kleopatra.
2. Click on Settings, then Configure Kleopatra. You want to add a keyserver. If Kleopatra doesn't automatically fill everything in for you, use the following settings:
 - Scheme: hkp
 - Server Name: keys.gnupg.net
 - Server Port 11371
 - Check the box labeled "OpenPGP"
3. Click Apply and close that window.
4. Click "Lookup Certificates on Server" and in the new window search for "signing@hashcat.net"
5. Look to the Key-ID field and make sure it says "8A16544F." Click on that entry once and then click "Import."
6. Back at the main Kleopatra window, right-click on the new key entry and select "Change Owner Trust..."
7. Download hashcat and the corresponding signature.
8. Open up Windows Explorer and navigate to your downloads directory. Right-click on the hashcat archive and mouse over "More GpgEX options," then click "Verify." A new window will pop up. Verify that the input file is the .asc signature you downloaded and that "Input file is a detached signature" is checked. If it all looks good, click on "Decrypt/verify." Once the scare warning and focus on the Key ID. If it says "0x8A16544F" then congratulations, you just verified the signature correctly.

Is there a hashcat GUI?

There are third-party graphical and web-based user interfaces available. The most up-to-date one is this: <http://www.hashkiller.co.uk/hashcat-gui.aspx>

We neither develop nor maintain these tools, so we can not offer support for them. Please ask the authors of the software for support or post questions on the forums you got the software from.

The main reason why there is no GUI developed by hashcat.net is because we believe in the power and flexibility of command line tools and well... *hashcat is an advanced password recovery tool (and being able to use the command line should be a bare minimum requirement to use this software).

Installation

How do I install hashcat?

There is no need to really install hashcat or hashcat legacy (CPU only version). You only need to extract the archive you have downloaded.

Please note, your GPU must be **supported** and the **driver must be correctly installed** to use this software.

If your operating system or linux distribution does have some pre-build installation package for hashcat, you may be able to install it using those facilities. For example, you can use the following under Kali Linux:

```
$ sudo apt-get update && sudo apt-get install hashcat
```

and update it with:

```
$ sudo apt-get update && sudo apt-get upgrade
```

Even if this is supported by some distributions, we do not directly support this here since it depends on the package maintainers to update the packages, install the correct dependencies (some packages may add wrappers, etc), and use reasonable paths.

In case something isn't working with the packages you download via your package manager, we encourage you to just download the hashcat archive directly, enter the folder, and run hashcat. This is the preferred and only supported method to "install" hashcat.

How does one install the correct driver for the GPU(s)?

Always make sure you have downloaded and extracted the newest version of  hashcat first.

If you have already a different driver installed than the recommended from the before mentioned download page, make sure to uninstall it cleanly (see [I may have the wrong driver installed. What should I do?](#)).

At this time you need to install the proprietary drivers for hashcat from [nvidia.com](https://www.nvidia.com) and [amd.com](https://www.amd.com) respectively. Do not use the version from your package manager or the pre-installed one on your system.

There is a detailed installation [guide for linux servers](#). You should prefer to use this specific operating system and driver version because it is always thoroughly tested and proven to work.

If you prefer to use a different operating system or distribution, you may encounter some problems with driver installation, etc. In these instances, you may not be able to receive support. Please, always double-check if AMD or NVidia do officially support the specific operating system you want to use. You may be surprised to learn that your favorite Linux distribution is not officially supported by the driver, and often for good reasons.

GPU device not found, why?

- Ensure you have the precise driver version recommended on <https://hashcat.net/hashcat/>
- Only **hashcat** supports cracking with GPU(s) (and OpenCL compatible CPU). hashcat legacy uses CPU but does not use your GPU, so there is no driver requirement for hashcat legacy
- Install the drivers directly from [nvidia.com](https://www.nvidia.com) or [amd.com](https://www.amd.com) for hashcat. Never use drivers provided by an OEM, Windows Update, or your distribution's package manager
- Make sure you download the correct driver: double check that version number and architecture (32 vs 64bit) match with your setup
- Make sure to stick exactly to the version noted on the hashcat page. It is ok to use a newer driver **only** if the hashcat page explicitly says "or higher."
- Development tools like CUDA-SDK or AMD-APP-SDK conflict with the drivers. Do not install them unless you know what you are doing!
- If you already have a conflicting driver installed, see [I may have the wrong driver installed. What should I do?](#)
- On AMD + Linux you have to configure `xorg.conf` and add all the GPU devices by hand. Alternatively, just run:

```
amdconfig --adapter=all --initial -f
```

and reboot. It is recommended to generate an xorg.conf for Nvidia GPUs on a linux based system as well, in order to apply the kernel [timeout patch](#) and enable fan control

I may have the wrong driver installed, what should I do?

(short URL: <https://hashcat.net/faq/wrongdriver>)

1. Completely uninstall the current driver
 - Windows: use software center
 - Linux:
 - NVIDIA: nvidia-uninstall
 - AMD: amdconfig --uninstall=force
 - If you installed the driver via a package manager (Linux), then you need to remove these packages too
 - Make sure to purge those package, not to just uninstall them
2. Reboot
3. For Windows only: download and start Driver Fusion (free version is enough; select "Display", AMD/NVidia/Intel, ignore the warning about Premium version), then Reboot
4. Make sure that no Intel OpenCL SDK, AMD-APP-SDK or CUDA-SDK framework is installed – if it is installed, uninstall it!
5. For Windows only: manually delete remaining OpenCL.dll, OpenCL32.dll, OpenCL64.dll files on all folders. You should find at least 2. They usually reside in "c:\windows\syswow64" and "c:\windows\system32". This step is very important!
6. For Linux only:
 - dpkg -S libOpenCL to find all packages installed that provide a libOpenCL, then purge them
 - find / -name libOpenCL* -print0 | xargs -0 rm -rf
7. Reboot
8. For Linux only: apt-get install ocl-icd-libopencl1 opencl-headers clinfo
9. Install the driver recommended on <https://hashcat.net/hashcat/>. If it says *exact* it means exact.
 - For AMD GPUs on Linux, see ROCm instructions [here](#).
10. Reboot
11. For Linux only: rm -rf ~/.hashcat/kernels
12. Reinstall hashcat, choose:
 - Stable version: Download and extract (under Linux, make sure to use: "7z x" to extract) the newest hashcat from <https://hashcat.net/>
 - Beta version: <https://hashcat.net/beta/>
 - Development version: git clone <https://github.com/hashcat/hashcat>
13. For Linux only: try to run "clinfo" first in your terminal
14. Try to run hashcat --benchmark

What does the cuModuleLoad() 209 error mean?

Official SDK ErrorCode: CUDA_ERROR_NO_BINARY_FOR_GPU = 209

This error can simply mean that the version of the driver that you have installed is too old:

- Solution: [How does one install the correct driver for the GPU\(s\)?](#)

However, it can also be worse than just that. The 209 error means that there was no precompiled kernel found in an existing precompiled kernel container file (the ones that end with .ptx). This sometimes happens with new GPUs that are not yet supported by hashcat.

- Solution: Please update your ForceWare driver to the latest one (not the recommended one)

If the problem still exist:

- Solution: Open an issue on [github](#). We will add support for your GPU with the next release.

What does the cuModuleLoad() 301 error mean?

Official SDK ErrorCode: CUDA_ERROR_FILE_NOT_FOUND = 301

You've unpacked the hashcat.7z archive using the command "7z e". This destroys the directory structure.

- Solution: Please use the **x** flag to decompress 7z archives. For example: "7z x"

NVidia added a completely new ShaderModel which is not yet support by hashcat.

- Solution: Please update your ForceWare driver to the latest one (not the recommended one)

If the problem still exist:

- Solution: Open an issue on [github](#). We will add your GPU with the next release.

What does the clGetPlatformIDs() -64 error mean?

This is the typical error message you get when you're AMD driver installation is faulty. You need to reinstall the recommended driver (while making sure that there is no older driver libs conflicting with it).

- Solution: [I may have the wrong driver installed, what should I do?](#)

What does the clGetPlatformIDs() -1001 error mean?

This is the typical error message you get when your AMD driver installation is faulty. You need to reinstall the recommended driver.

- Solution: [I may have the wrong driver installed, what should I do?](#)

What does the clGetDeviceIDs() -1 Error error message mean?

Receiving this error means that hashcat could not detect any capable GPUs in your system. If you are positive that you have capable GPUs you can try the following suggestions to fix your system:

- #define CL_DEVICE_NOT_FOUND -1
- Are you logged in as the user who has opened the X session?
- Did you use "export DISPLAY=:0"?
- Do you have X Authority and/or disable X11 ACLs with "xhost +"
- Is your display manager configured correctly? (gdm example config: <http://www.sch0.org/gdm.conf>)
- Did you use the recommended driver (for the hashcat version you use) mentioned on the download page? [I may have the wrong driver installed, what should I do?](#)

What does the clBuildProgram() -11 error mean?

Official SDK ErrorCode: CL_BUILD_PROGRAM_FAILURE -11

This means you are using an incompatible driver version. Sometimes the structure inside the .llvmlr archive changes. This is something the developers of hashcat have no influence on. This structure is dictated by the driver.

- Solution: Make sure you have the latest hashcat version
- Solution: [I may have the wrong driver installed, what should I do?](#)

What does the clCreateBuffer() -61 error mean?

Official SDK ErrorCode: CL_INVALID_BUFFER_SIZE -61

This is the typical "Out-Of-Memory" Error, and is not driver related. Note that this refers to your GPU memory, not host memory. Also note that there is a maximum allocation size which is typically only 256MB. That means even if your GPU has 4GB ram, hashcat is allowed only to allocate 256MB per buffer.

Solution:

- Reduce -n
- Reduce -w
- Reduce number of hashes

What does the clEnqueueCopyBuffer() -30 error mean?

Official SDK ErrorCode: CL_INVALID_VALUE -30

This is not a driver related error. This error occurs mostly in cases where you are using too many rules. You can adjust the following parameters to reduce your total rule count:

- decrease -g value
- decrease number of -r stacks

What does the cuStreamSynchronize() 702 error mean?

This is the same error type as OpenCL -30 Error. It is not a driver related error. This error occurs mostly in cases where you are using too many rules. You can adjust the following parameters to reduce your total rule count:

- decrease -g value
- decrease number of -r stacks

What does the error: FanSpeed_Get(): -5 mean?

Official ADL SDK ErrorCode: ADL_ERR_INVALID_ADL_IDX -5

The most likely reason of this problem is that hashcat could not perfectly map the found and supported OpenCL devices to the devices that the ADL (AMD Display Library) did find. The main problem here is that ADL and OpenCL do use some different means to describe/priorize/sort/name/identify the different devices.

hashcat does some fuzzy matching to try matching those devices with the best accuracy possible, but still sometimes this fails. It could, for instance, fail when you have a multi-GPU setup where some devices are identical (same model/make etc), and you specify -d 1/-d 2.

There is little hashcat can do to avoid this problem since the bus id and device id are sometimes the same (reported by OpenCL), even if we are dealing with 2 "different" (but similar) GPUs.

A possible workaround is to set the fan manually to 100% and use --gpu-temp-disable flag in hashcat.

What does the ADL_Overdrive5_Temperature_Get(): -1 error mean?

Official ADL SDK ErrorCode: ADL_ERR -1

The most likely reason for this problem is that hashcat could not perfectly map the found and supported OpenCL devices to the devices that the ADL (AMD Display Library) did find. ADL and OpenCL do use some different means to describe/priorize/sort/name/identify the different devices, which can contribute to this problem.

hashcat does some fuzzy matching, due to the lack of a unique and consistent way to identify the devices, to try to match those devices with the best accuracy possible, but still sometimes this fails. It could, for instance, fail when you have a multi-GPU setup where some devices are identical (same model/make etc), and you specify -d 1/-d 2.

There is little hashcat can do to avoid this problem since the bus id and device id are sometimes the same (reported by OpenCL), even if we are dealing with 2 "different" (but similar) GPUs.

In this particular case, hashcat fails to get the temperature of the GPU which uses overdrive 5. This could be because the mapping somehow failed and the device is an overdrive 6 GPU instead or something similar.

A possible workaround is to set the fan manually to 100% and use the --gpu-temp-disable flag in hashcat.

I got this error message: undefined symbol: nvmlInit_v2 Error, what does it mean?

This is simple. You are using a version of the Nvidia ForceWare driver that is way too old. It is time to update!

- Solution: [How does one install the correct driver for the GPU\(s\)?](#)

What does the clEnqueueNDRangeKernel() : -4 : CL_MEM_OBJECT_ALLOCATION_FAILURE error mean?

This is a typical error with AMD GPUs when using the packaged driver (*.deb package) as "/usr/lib/libOpenCL.so.1" is exchanged by the package.

Solution:

- completely purge fglrx
- apt-get install --reinstall ocl-icd-libopencl1
- service xdm stop
- install driver (not package version: e.g. radeon-crimson-15.12-15.302-151217a-297685e.zip)
- reboot
- service xdm stop
- amdconfig --initial -f --adapter=all
- reboot

How can I use old hashcat/oclHashcat versions?

We really encourage you to always use the latest and greatest version of hashcat. In general, it comes with several improvements, problem fixes, new features, better GPU support etc. In our opinion, there is only one valid reason to use older versions of hashcat: when AMD/Nvidia dropped support for your graphics card and hashcat developers decided that it is more appropriate to support the newer generation cards (and hence support/optimize code for the more recent driver versions) instead of keeping support for no-longer supported GPUs.

Post-Installation

I see only the usage, it seems to be a syntax error. What should I do?

First of all, look closely at the usage (see the Usage: line at the beginning of the --help output). It should say something like this:

For hashcat:

```
Usage: hashcat [options]... hash|hashfile|hccapxfile [dictionary|mask|directory]...
```

For hashcat legacy:

```
Usage: hashcat [options] hashfile [mask|wordfiles|directories]
```

This means for instance that the hash file comes before the word list, directory or mask. Swapping these parameters, e.g. hash file after the mask, is not supported.

If this is not the syntax error you have experience, please have a closer look at the --help output and search the [wiki](#) for more details about the syntax.

hashcat unpacks to 1GB or more, do I really need all the files?

It depends on your system, but usually you do not need all the files within the extracted hashcat folder. The most space is taken from the precompiled kernel files. There's a kernel for each GPU type. When hashcat starts up it queries model from your GPU and loads the corresponding kernel for it. This ensures maximum performance.

Depending on if you have a Nvidia or AMD system, the bitness is also important. For Nvidia systems there are kernels for 32 bit and 64 bit. So if you use a 64 bit system you don't need the 32 bit kernels at all and you can safely remove them.

But there is more. It's unlikely that you have all GPU types in a single system. You can safely remove those kernel files which correspond to GPUs that you do not have in your system. To find out which GPU you have, you can run:

```
$ ./hashcat.bin 00000000000000000000000000000000
...
Device #1: Kernel ./kernels/4318/m00000_a0.sm_52.64.ptx
Device #2: Kernel ./kernels/4318/m00000_a0.sm_50.64.ptx
Device #3: Kernel ./kernels/4318/m00000_a0.sm_21.64.ptx
...
```

On this system I have 3 graphic cards installed. One uses sm_21, one uses sm_50 and one uses sm_52. So I can safely remove all kernel that do not match this list (but only those with sm_xx within the file name for Nvidia).

On AMD systems it's a bit different. They are distinguished by either VLIW1, VLIW4 and VLIW5. Basically you can say that all GCN (hd7970, R9 cards) cards are VLIW1. VLIW4 is for the 6xxx series and VLIW5 for the 5xxx series. That means, for example, you can typically remove all VLIW4 and VLIW5 kernel if you only have a hd7970 in your system (but only remove those kernels which have the VLIWx within the file name for AMD).

Basic cracking knowledge

What does "Status: Exhausted" mean?

Exhausted simply means hashcat has tried every possible password combination in the attack you have provided, and failed to crack 100% of all hashes given. In other words, hashcat has finished doing everything you told it to do – it has *exhausted* its search to crack the hashes. You should run hashcat again with a different attack/word list/mask etc.

What is a hashcat mask file?

A [hashcat mask file](#) is a set of masks stored in a single plain text file with extension .hcmask.

The format of the lines is defined here: [hashcat mask file](#)

Each line of a .hcmask file could contain this information (the order of the fields is exactly as mentioned here):

- --custom-charset1 (short -1, optional): define a custom char set which you can reference in the mask with ?1
- --custom-charset2 (short -2, optional): define a custom char set which you can reference in the mask with ?2
- --custom-charset3 (short -3, optional): define a custom char set which you can reference in the mask with ?3
- --custom-charset4 (short -4, optional): define a custom char set which you can reference in the mask with ?4
- mask (required): specify the mask

Each and every field mentioned above is separated by a comma (",", without quotes). If an optional field is not specified, the comma does not need and can't be written down, hence if you only want to specify a mask (without the optional custom char sets), the lines should look like this:

```
?u?l?l?l?l?s
?u?l?l?l?l?s
?l?l?l?l?l
?l?l?l?l?l?s
```

Note: all the custom char sets were not set here (-1, -2, -3, -4), nor used (?1, ?2, ?3, ?4), hence we only have 1 single field (and hence no commas).

On the other hand, if you for instance only need --custom-charset1 (or short -1), your lines would look something like this:

```
?l?d,?l?l?l?l?l
?l?u,?l?l?l?l?l?l
```

Note: here the --custom-charset1 would be set to ?l?d with the first hashcat mask file line and to ?l?u with the second line (those 2 lines are independent). Also note that when the --custom-charset1 (or short -1) field was set, it should also be used within the mask with ?1. The opposite is also true, when you use a custom char set within the mask (?1) you should also set the --custom-charset1 (or short -1) field to a valid value.

If 2 custom char sets are needed, you would use something like this:

```
?l?d,?u?l,?2?l?l?l?l?l?l
```

Note: here we set --custom-charset1 (or short -1) to ?l?d and --custom-charset2 (or short -2) to ?u?l and then use those custom char sets within the mask.

... and so on and so forth (up to the 4 supported custom char sets).

There are 4 important syntax rules:

- if you use \, the comma will be seen by *hashcat as a literal character, i.e. the backward slash escapes the comma (which would otherwise be seen as a separator between the different fields: -1, -2, -3, -4 and the mask)

- if you use # at the beginning of the line, the line will be treated as a comment and hence ignored
- if you use \# at the beginning of the line, the # will be used literally as a character (since the backward slash escapes the # which otherwise would be seen as a comment)
- if you want to use a question mark ("?", without quotes) within your mask or within your custom charsets, you need to escape it with an additional question mark, i.e. ?? means that the question mark is used as a literal symbol

You can **use** .hcmask files as simple as this:

```
$ ./hashcat.bin -m 0 -a 3 md5_hash.txt my.hcmask
```

In other words, you simply specify the path to the .hcmask file at the position in the command line where you normally would use the single mask.

What exactly is a weak hash?

The weak hash detection is a special feature of hashcat. The goal of this feature is to notice if there is a hash whose plaintext is empty, that means a 0-length password. Typically when you just hit enter. We call it a weak-hash check even if it should have been called weak-password check but there are simply too many weak-passwords.

However, if your hashlist contains millions of salts we have to run a kernel for each salt. If you want to check for empty passwords for that many salts you will have a very long initialization/startup time by running hashcat. To work around this problem there is a parameter called "--weak-hash-threshold". With it you can set a maximum number of salts for which weak hashes should be checked on start. The default is set to 100, that means if you use a hashlist with 101 unique salts it will not try to do a weak-hash check at all. Note we are talking about unique salts not unique hashes. Cracking unsalted hashes results in 1 unique salt (an empty one). That means if you set it to 0 you are disabling it completely, also for the unsalted hashes.

What is a potfile?

The potfile stores which hashes were already cracked, and thus won't be cracked again.

The reason for this is that otherwise e.g. the output file (--outfile, -o) could contain the same and identical hash output again and again (if different attack types lead to the same password candidates which do match).

It also has an enormous effect on cracking salted hashes. If hashcat notices that all hashes which are bound to a specific salt are cracked, it's safe to not generate new guesses for this specific salt anymore. This means, for example, if you have 2 hashes with different salts and one is cracked, the speed is doubled. Now if you restart the session for any reason the potfile marks the one cracked hash as cracked and so the salt is marked as cracked. You startup with doubled guessing speed.

You can disable potfile support completely by using --potfile-disable. However we strongly recommend leaving it enabled. If you have a large list of salted hashes for example and you do not use --remove and for whatever reason you have to restart this cracking session all your bonus guessing speed is loss.

Note that using a potfile is very different from the idea which you have in mind when you are used to use --remove. Having a hashlist with only uncracked hashes is fine, but with potfile you can do the same if you use the --left switch. For example, if your cracking session is finished and you want to have a left list, you simply run:

```
$ ./hashcat.bin --left -o leftlist.txt -m 0 hash.txt
```

Now you have both, the original list and the left list.

It's also safe to copy (or append) the data from one potfile to another.

The potfile is stored in hashcat's profile folder with the name "hashcat.potfile". You can override this path by using the --potfile-path parameter.

How can I identify the hash type?

There is no concrete method for identifying a hash algorithm using the hash alone.

Some hashes have signatures which give a strong indication of which algorithm was used, such as "\$1\$" for md5crypt. Usually you can rely on this information; however, this method of identification is not bullet-proof! For example, consider an algorithm such as crypt(sha256(pass), "\$1\$").

For hashes which have no signature, it is virtually impossible to distinguish which algorithm was used. A string of 32 hex characters could be LM, NTLM, MD4, MD5, double MD5, triple md5, md5(sha512(pass)), so on and so forth. There is literally an infinite number of possibilities for what the algorithm may be!

Tools which claim to be able to identify hashes simply use regular expressions to match the hash against common patterns. This method is extremely unreliable and often yields incorrect results. It is best to avoid using such tools.

A much better way to identify the hash algorithm would be to understand the origin of the hashes (e.g. operating system, COTS application, web application, etc.) and make an educated guess at what the hash algorithm might be. Or better yet, use the source!

For some example hashes see the [example hashes](#) wiki page.

Hashcat reports "Status: Cracked", but did not print the hash value, and the outfile is empty. What happened?

If you see the line "INFO: removed X hashes found in pot file", where X could be any number greater than 0, it means that you had already cracked the target hash during a previous run of this session. The hash will not be cracked again for the following reasons:

- you already cracked the hash, hence cracking the same hash would be a waste of resources;
- the potfile has already all the information to be able to show the password (use --show);
- potfile support wasn't disabled, thus a check for every hash in your hash file (or the one hash in the command line) will be performed. If it was found in the .potfile, it will be marked as already cracked and will not be loaded by hashcat

The reason for not showing/storing the same crack is that it is a waste of resources to crack a single hash more than once and hashcat assumes that if it was already cracked, the user did already "see" the password. The potfile-reading feature (at startup) will check if some hashes within the hash list were already cracked and marked them as "already cracked" without trying to crack them again and again.

It is possible to show all the cracks with --show and this will show every previous cracks (stored within the .potfile). The path to the .potfile can be specified by the --potfile-path command line argument. This way it is possible that you use different .potfile files for each and every different hash list (the default name is "hashcat.potfile").

It is possible to avoid the warning and the storage of all cracks by using --potfile-disable.

How can I show previously cracked passwords, and output them in a specific format (e.g. email:password)?

To accomplish this, you need to use the --show switch.

--show is always used **after** the hashes have been cracked; therefore, you only need (and should **not** specify more than) these command line arguments:

- --show (required)
- the hash type (optional, i.e. -m 0 is the default one)
- the path to the pot file (--potfile-path, optional): specify a specific input .potfile (containing previous cracks)

- user name (--username, optional): tell *hashcat that the hash file has user names / email addresses in the first field of each and every line
- the output file (--outfile, -o, optional): file to store the output
- the output format (--outfile-format, see --help for the possible values, optional): changes the way the output is shown, e.g. password only, password in hexadecimal, don't show hash etc
- quiet mode (--quiet, optional): enables the quiet mode (does not show any extra info)
- the original hash file (required): this needs to be the unmodified hash list, --remove does modify / delete hashes from the hash file

Example:

```
$ ./hashcat.bin -m 0 --show -o formatted_output.txt --outfile-format 2 original_MD5_hashes.txt
```

Note: You do not need - and should not specify - any mask, dictionary, wordlist directory, etc.

An example how this 2-fold process would look like is shown below:

1. crack the hashes:

```
$ ./hashcat.bin -m 0 --username --potfile-path md5.potfile original_MD5_hashes.txt rockyou.txt
```

2. output the hashes to a file with --show:

```
$ ./hashcat.bin -m 0 --username --potfile-path md5.potfile --show -o formatted_output.txt --outfile-format 2 original_
```

What is a keyspace?

Keyspace is the term used to refer to the number of possible combinations for a specified attack. In hashcat, it has a special meaning that is not exactly the same as the usual meaning. The output of --keyspace is designed to be used to distribute cracking, i.e. you can use the value from --keyspace and divide it into x chunks (best would be if the chunk size depends on the performance of your individual nodes if they are different) and use the -s/-l parameters for distributed cracking.

To tell devices which candidates to generate on GPU, hashcat keeps track of some of the candidates on the host. To do this, there are two loops: a loop that runs on the host (the "base loop"), and a loop that runs on the device (the "mod(ifier) loop.")

To work between multiple compute nodes, hashcat must divide up and distribute portions of the base loop. This is where --keyspace, -s, and -l come into play. --keyspace reports the size of the base loop that executes on the host so that we know how to divide up the work. -s and -l control the start/stop positions of the base loop.

In other words, **hashcat's --keyspace is specifically designed to optimize distribution of work, and is not a literal representation of the total possible keyspace for a given attack.**

The keyspace is calculated according to the following formulas for each program:

hashcat legacy

- -a 0 - number of words in wordlist
- -a 0 + rules - number of words in wordlist multiplied by number of rules in rule file
- -a 1 - number of words in wordlist multiplied by 2
- -a 1 + rules - number of words in left_wordlist multiplied by number of words in right_wordlist multiplied by number of rules in rule file
- -a 3 - multiply the number of possible characters in each mask position together
- -a 5 - number of words in wordlist multiplied by number of entries in table
- -a 8 - it's complicated

hashcat

- -a 0 - number of words in wordlist
- -a 0 + rules - number of words in wordlist
- -a 1 - number of words of the larger word list out of the 2 word lists specified (left_wordlist, right_wordlist)
- -a 3 - multiply the number of possible characters in each mask position for the base loop mask, excluding the mod loop mask - please just use --keyspace switch ;)
- -a 6 - number of words in wordlist (the left part, the dictionary, is most significant)
- -a 7 - multiply the number of possible characters in each mask position (the left part, the mask, is most significant)
- -a 9 - not really applicable, or "one" - only one candidate is applied to one hash
- -a 9 + rules - not really applicable - basically just the number of rules in rule file

You can calculate the keyspace for a specific attack with the following syntax:

```
$ ./hashcat.bin -m 0 -a 0 example.dict -r rules/best64.rule --keyspace
129988
```

(Don't include the hashfile!)

See also [this forum post](#).

How to deal with special non-latin characters (chinese, arabic, etc...) ?

Various languages use different character encodings. It may seem overwhelming that there are many different encoding types, many languages, and many different characters that exist. What you need to know when it comes to encoding is that most, if not all, hashing algorithms do not care about encoding at all. Hash algorithms just work with the single bytes a password is composed of. That means if you input a password that contains, for example, a German umlaut, this can result in multiple different hashes of the same unsalted algorithm. To further illustrate this, you will see three different hashes depending on whether you have used ISO-8859-1, utf-8 or utf-16.

There's no built-in character conversation in hashcat, but this doesn't mean you can not crack them:

So you need to solve this problem outside of hashcat / hashcat legacy. An easy solution would be to simply convert your wordlists with iconv:

```
$ iconv -f utf-8 -t iso-8859-1 < rockyou.txt | sponge rockyou.txt.iso
```

When using Brute Force, if you're fine to stick to ISO/codepage encodings there are special hashcat charsets that can be found in the charset/ folder:

```
.
./special
./special/Slovak
./special/Slovak/sk_cp1250-special.hcchr
./special/Slovak/sk_ISO-8859-2-special.hcchr
... etc
```

Note: **hashcat charset files** (.hcchr) can be used like any other **custom charsets** (--custom-charset1, --custom-charset2, --custom-charset3, --custom-charset4).

But note, nowadays a lot of sources use utf-8. This makes things a bit more complicated.

Here's a nice blog-post of how to deal with utf-8 and utf-16 with hashcat legacy and hashcat: <http://blog.bitcrack.net/2013/09/cracking-hashes-with-other-language.html>

Most of this is about using --hex-charset or --hex-salt where you can define everything in hex. In the end, all character encodings will fall to this.

Unfortunately there is no bullet-proofed way to know if a specific hash (or hash list) uses a specific encoding. The most straight-forward way would be to just try and crack some hashes with the encoding you think is most likely. But this could fail of course when you try with a very "different" encoding. To see if hashcat does indeed run the correct password candidates you want it to run, you can just create some example hashes and try to crack them with for instance a Dictionary-Attack or with a mask attack by using .hchr files or --hex-charset.

Attack-Modes

Why should I use a mask attack? I just want to "brute" these hashes!

Read [Mask attack](#). A mask attack is a modern and more advanced form of "brute force".

It can fully replace brute force, and at the same time mask attacks are more flexible and more capable than traditional brute force attacks.

The general options you should consider/need are:

1. --increment (optional): specifies that the length of the password candidates shouldn't be fixed, but increase in length
2. --increment-min (optional): the minimum length for --increment (if --increment is used but --increment-min was not set, it defaults to 1)
3. --increment-max (optional): the maximum length for --increment (if --increment is used but --increment-max was not set, it defaults to the length specified by the mask, see #4)
4. --custom-charset1 (-1), --custom-charset2 (-2), --custom-charset3 (-3), --custom-charset4 (-4) (all optional): you can define custom charsets e.g. all lower letters together with all upper letters, plus all digits: --custom-charset1 ?u?l?d (Attention: this is just the definition of a custom charset, you need to **use** these custom charset definitions within the mask)
5. mask (required): specifies the charsets (could be **built-in** or **custom** ones) and implicitly also the maximum length of the password candidates. If --increment option is not used, it specifies the fixed length (because min and max length would be set to the length implied directly from the mask length itself)

An example command would therefore look something like this:

```
$ ./hashcat.bin -m 0 -a 3 --increment --increment-min 4 --increment-max 6 hash.txt ?a?a?a?a?a?a?a
```

Explanation:

- -m 0: we set the hash type to MD5 (see [Example hashes](#))
- -a 3: set the attack mode to mask attack (see [Mask attack](#))
- --increment: enable incremental mode (see #1 above)
- --increment-min 4: set the minimum length of the password candidates to 4 (in this case)
- --increment-max 6: set the maximum length of the password candidates to 6 (in this case)
- ?a?a?a?a?a?a?a: the mask is a 8 character long string of the built-in charset ?a ("all", which includes lower and upper case characters, digits and special characters)

Note that even if the mask is of length 8 in this particular example the passwords candidates are limited by --increment-min and --increment-max and hence are of length 4 to 6.

If --increment-max 6 was not specified, the maximum length would be implicitly set to 8 since the mask itself is of length 8.

I do not know the password length, how can I increment the length of the password candidates?

You can use --increment (or short -i), --increment-min and --increment-max.

Make sure that the mask (which should always be set, is required) is at least the same length of the --increment-max value or the maximum password candidate length you want to try (if --increment-max was not specified). By the way, the value for --increment-max should also not be greater than the length of the mask, i.e. the main limiting factor is the mask length, after that --increment-max, if specified, will further limit the length of the password candidates.

- Examples of correct commands:

```
./hashcat.bin -m 0 -a 3 -i ?d?l --increment --increment-min 5 md5_hash.txt ?l?l?l?l?l?l?l?l
```

Note: the limiting length is set by the mask (?l?l?l?l?l?l?l?l). Therefore you can think of this command as if there was an automatically added --increment-max 8. This means you do not need to specify --increment-max 8 if it can be automatically determinate by the mask length.

```
./hashcat.bin -m 0 -a 3 -i --increment-min 2 --increment-max 6 md5_hash.txt ?a?a?a?a?a?a?a
```

Note: here --increment-max was indeed set to a value **less** than the mask length. This makes sense in some cases where you do not want to change the mask itself, i.e. leave the 8 position long mask as it was (?a?a?a?a?a?a?a).

```
./hashcat.bin -m 0 -a 3 -i --increment-min 6 --increment-max 8 md5_hash.txt ?a?a?a?a?a?a?a
```

Note: it is even possible to set the --increment-max value to the same length of the mask even if the --increment-max value would be implied anyway by the mask length.

```
./hashcat.bin -m 0 -a 3 -i --increment-max 6 md5_hash.txt ?l?l?l?l?l?l?l?l
```

Note: also --increment-min must not necessarily be set, if skipped it will start with length 1 (and if --weak-hash-threshold 0 was not set it will even start with length 0).

- Examples of **incorrect** commands and reasons why they are incorrect:

Attention: these are commands that should not be used, they do not work (there only purpose is to show you what is not accepted)

```
./hashcat.bin -m 0 -a 3 --increment --increment-max 8 md5_hash.txt ?a
```

Note: this is the most common user error, i.e. the user did not understand that the winning limiting factor is **always** the mask length (here length 1). Even if --increment-max 8 was specified, the mask is too short and therefore hashcat can't increment that mask. The reason why is simple: mask attack is a per-position attack, each position can have its own charset. There is a strict requirement that the user specifies the charset for each position. If the custom or built-in charset was not specified for the (next) position, hashcat can not know what should be use as a charset and hence stops at the position where it was still clear what charset should be used (in this example it is length 1). The decision to stop and to refrain to imply charsets is made by the developers on purpose because otherwise (if hashcat would silently and magically determine a "next implied charset") there could be strange errors and unexpected behavior.

```
./hashcat.bin -m 0 -a 3 --increment --increment-min 2 --increment-max 3 md5_hash.txt ?a
```

Note: also here the value of --increment-max is not within the length of the mask. In addition, also --increment-min is incorrect here because its value is outside of the bounds too.

```
./hashcat.bin -m 0 -a 3 --increment --increment-min 6 --increment-max 10 md5_hash.txt ?a?a?a?a?a?a?a
```

Note: always make sure that the length of the mask (in this case ?a?a?a?a?a?a?a) is long enough, in this case it must be at least of length 10 (it is ?a?a?a?a?a?a?a?a?a?a).

```
./hashcat.bin -m 0 -a 3 --increment --increment-min 4 --increment-max 3 md5_hash.txt ?a?a?a?a?a?a?a
```

Note: the value of --increment-min must always be less or equal to the value of --increment-max. This is not satisfied here since 4 > 3.

For more details about mask attack see [Why should I use a mask attack? I just want to "brute" these hashes!](#)

I want to optimize my Brute-Force attack by ordering letters by frequency in a custom charset. How to do it? Does it make sense?

That's clever, however note that hashcat uses markov-chain like optimizations which are (in theory) more efficient. You need to disable this feature to force hashcat to accept your special ordering. This can be done using --markov-disable parameter.

I want to use rules, but there are three different parameters. When do I use -r, -j and -k?

For the majority of times using "-r rulefile" is the one you want. It is used in a straight attack -a0 to manipulate a dictionary and will load one or more rule files containing multiple rulesets.

If you are using combinator or hybrid attacks you can use -j and -k to manipulate either the left or the right side of the input.

For example a combinator attack that toggles the first character of every input from the leftside.txt:

```
$ ./hashcat.bin -a 1 example0.hash leftside.txt rightside.txt -j T0
```

How does one use several rules at once?

Firstly, we need to distinguish 2 different cases:

1. the rules you want to use should be applied: independently from each other, not chained/concatenated, but applied one after the other
2. the rules you want to use should be applied: together as if they were (previously) combined with combinator.bin, e.g. all rules from the first .rule file are combined with all rules from the second .rule file

For case number 1 you can just "cat" the individual files

Note: it would be better to use some kind of duplicate removal instead, e.g. sort -u, but note also that sort -u does not necessarily remove all duplicates since the rule syntax allows for extra spaces and furthermore a set of 2 different rules may lead to similar or identical plains in some or all situations because it is possible that combination of different rules could be considered as identical even if they are not identical when comparing the rule "text"/strings.

The following description will deal only with case number 2 (the rules should be chained, applied at the same time).

hashcat allows for rule stacking. This can easily be achieved just by appending more rule files to your attack.

Note: hashcat legacy does not support stacking. Only a single -r parameter is permitted.

```
$ ./hashcat.bin -a 0 -m 0 -r rules/best64.rule -r rules/toggles2.rule hashlist.txt dict.txt
```

Note: depending on the rules themselves, the order of the different -r arguments might be very important. You may need to double-check which -r parameter is the first one on the command line (this will be applied first), which should be the second one (this will be applied next), etc ...

OK, there is a hybrid attack for append mask and prepend mask, but what if I want to use both at the same time?

To do this, you can use the rule-stacking feature of hashcat: [How does one use several rules at once?](#)

For example, if you want to do something like ?d?dword?d?d, that is two digits both appended and prepended you can do the following:

```
$ ./hashcat.bin hash.txt wordlist.txt -r rules/hybrid/append_d.rule -r rules/hybrid/append_d.rule -r rules/hybrid/prep
```

Such rules exist for all the common charsets.

You can easily create your own hybrid rules using maskprocessor: [rules_with_maskprocessor](#)

When I use --increment in hybrid attack how does that work?

Given an attack

```
$ ./hashcat.bin -a 6 example0.hash example.dict ?a?a?a?a?a --increment
```

Hashcat iterates through the given mask until the full length is reached.

```
Input.Left.....: File (example.dict)
Input.Right.....: Mask (?a) [1]
Hash.Target.....: File (example0.hash)
```

```
Input.Left.....: File (example.dict)
Input.Right.....: Mask (?a?a) [2]
Hash.Target.....: File (example0.hash)
```

```
Input.Left.....: File (example.dict)
Input.Right.....: Mask (?a?a?a) [3]
Hash.Target.....: File (example0.hash)
```

```
Input.Left.....: File (example.dict)
Input.Right.....: Mask (?a?a?a?a) [4]
Hash.Target.....: File (example0.hash)
```

```
Input.Left.....: File (example.dict)
Input.Right.....: Mask (?a?a?a?a?a) [5]
Hash.Target.....: File (example0.hash)
```

For more details about mask attack see [Why should I use a mask attack? I just want to "brute" these hashes!](#)

How to use multiple dictionaries?

If you use hashcat with a Dictionary attack (-a 0) you can specify several dictionaries on the command line like this:

```
$ ./hashcat.bin -m 0 -a 0 hash.txt dict1.txt dict2.txt dict3.txt
```

This list of wordlist is currently only allowed with -a 0 parameter. Note that this also works with so-called globbing (of shell parameters and in this case paths/file names), since your operating system/the shell expands the command lines to (among others) full file paths:

```
$ ./hashcat.bin -m 0 -a 0 hash.txt ../my_files/*.dict
```

Furthermore, if you want to specify a directory directly instead, you could simply specify the path to the directory on the command line:

```
$ ./hashcat.bin -m 0 -a 0 hash.txt wordlists
```

Note: sometimes it makes sense to do some preparation of the input you want to use for hashcat (outside of hashcat). For instance, it sometimes makes sense to sort and unique the words across several dictionaries if you think there might be several "duplicates":

```
$ sort -u -o dict_sorted_uniqued.txt wordlists/*
```

[hashcat utils](#) might also come handy to do some preparation of your wordlists (for instance the [splitlen](#) utility etc)

Why are there 5 different toggle rules in the rules/ folder?

You often hear the following: A great and simple way to make your password harder to crack is to use upper-case characters. This means you flip at least two characters of your password to upper-case. But note: don't flip them all. Try to find some balance between password length and number of upper-case characters.

We can exploit this behavior leading to an extreme optimized version of the original Toggle-case attack by generating only all these password candidates that have two to five characters flipped to upper-case. The real strong passwords have this balance, they will not exceed this rule. So we don't need to check them.

This can be done by specialized rules and since hashcat and hashcat legacy support rule-files, they can do toggle-attacks that way too.

See `rules/toggle[12345].rule`

Depending on the rule-name they include all possible toggle-case switches of the plaintext positions 1 to 15 of either 1, 2, 3, 4 or five 5 characters at once.

When I run an attack with -a 3 and I do not specifying a mask, I see it working but what is it doing?

The reason why there is no (syntax) error shown when you didn't specify any mask, is that hashcat/hashcat legacy have some [default values](#) for masks, custom charsets etc. This sometimes comes in very handy since the default values were chosen very wisely and do help some new users to get started very quickly.

On the other hand, sometimes this "feature" of having some default values might confuse some users. For instance, the default mask, for good reasons, isn't set to a mask consisting of the built-in charsets ?a or even ?b which some users might expect, but instead it is an optimized mask which should (in general) crack many hashes without covering a way too large keyspace (see the [default values](#) page for the current default mask).

This also implies that when you don't specify a mask explicitly, it could happen (and is very likely) that you do not crack some hashes which you might expect to be cracked immediately/easily (because of the reduced keyspace of the default mask). Therefore, we encourage you that you always should specify a mask explicitly to avoid confusion.

If you still do not know how to do so, please read [Why should I use a mask attack? I just want to "brute" these hashes!](#)

How does one use the new prince attack mode with hashcat legacy?

Luckily, with latest version of *hashcat legacy* the attack-mode is built-in. You can simply use it using the -a 8 selection. Do not forget to name a wordlist, like rockyou.txt or so.

For *hashcat* you need to use a pipe and the princeprocessor (standalone binary) from here:

<https://github.com/jsteube/princeprocessor/releases>

Then your simply pipe like this for slow hashes:

```
$ ./pp64.bin rockyou.txt | ./hashcat.bin -m 2500 -w 3 test.hccapx
```

In case you want to crack fast hashes you need to add an amplifier to achieve full speed:

```
$ ./pp64.bin rockyou.txt | ./hashcat.bin -m 0 -w 3 test.md5 -r rules/rockyou-30000.rule
```

I have a half-known password. I know the first 4 letters, can hashcat get the rest of the password?

Yeah that actually works, thanks to the mask attack! To understand how to make use of this information you really need to read and understand the mask-attack basics. So please read this article first: [Mask Attack](#)

Now that you've read the Mask Attack article it's easy to explain. For example consider that you know that the first 4 chars of the password are "Pass" and you know that there's like 3 or 4 or 5 more letters following from which you do not know if they are letters, digits or symbols you can do the following mask:

```
Pass?a?a?a?a -i
```

To understand how this works with the incremental, please also read this article:

[I do not know the password length, how can I increment the length of the password candidates?](#)

Why are there two different hash-modes for Vbulletin?

There are actually two different hash-modes for Vbulletin. Somewhere between version v3 and v4 they've changed the default salt-length from 3 characters to 30 characters. From a high-level programming language view this has no impact but from our low-level view this is really a difference. That's because of the block-modes used in many hashes, even in that case.

Vbulletin uses a scheme that is simply written like this: `md5(md5(pass).salt)`

So it first computes the md5 hash of the password itself and then it concatenates it with the salt. Since the software rely on PHP and the md5 function in PHP returns by default a ascii hex representation we have a total length in the final md5() transformation of length 32 + 30 = 62.

The problem here is that 62 > 55 and 55 is the maximum buffer for a single md5 transformation call. What we actually need to do now, from low-level perspective, is to compute the hash using the buffer of 62 and then compute another md5 with a buffer nearly empty. That's [RFC](#). That means for Vbulletin v3 we have to compute 2x md5 calls and for v4 we need 3x md5 calls while the scheme itself stayed untouched. In other words, from GPU kernel view this is a completely different algorithm and that's why they are two different hash-modes.

Performance

How much faster is cracking on Linux compared to a Windows operating system?

Not at all and that's true for both hashcat and hashcat legacy. Even the GPU drivers are equally good or bad (depends on how you see it).

IOW, if you feel comfortable with Windows and all you want to do is to crack hashes you can stick to Windows.

How can I perform a benchmark?

If you want to find out the maximum performance of your setup under ideal conditions (single hash brute force), you can use the built-in benchmark mode.

```
$ ./hashcat.bin -m 0 -b
hashcat (v6.1.1) starting in benchmark-mode...

...
Speed.#*.....: 15904.5 MH/s
...
```

This mode is simply a brute force attack with a big-enough mask to create enough workload for your GPUs against a single hash of a single hash-type. It just generates a random, uncrackable hash for you on-the-fly of a specific hash-type. So this is basically the same as running:

```
$ ./hashcat.bin -m 0 00000000000000000000000000000000 -w 3 -a 3 ?b?b?b?b?b?b?b

...
Speed.#*.....: 15907.4 MH/s
...
```

Please note the actual cracking performance will vary depending on attack type, number of hashes, number of salts, keyspace, and how frequently hashes are being cracked.

The parameters you should consider when starting a benchmark are:

- `--benchmark` (short `-b`, required): tell hashcat that it should perform a benchmark
- `--hash-type` (short `-m`, default is to benchmark several important algorithms, optional): tell hashcat which hash types should be benchmarked
- `--benchmark-mode` (default value is 1, value 0 means that you can tune with `-u -n`, optional): set the benchmark mode

This means, that for instance a command as simple as this:

```
$ ./hashcat.bin -b
```

will give you a list of benchmark results for the most common hash types available in hashcat (with performance tuning, `--benchmark-mode 1`).

My desktop lags too much, anything I can do to avoid it?

In order to give the GPU more breathing room to handle the desktop you can set a lower ("w 1") workload profile:

```
-w, --workload-profile=NUM      Enable a specific workload profile, see references below

* Workload Profile:

  1 = Reduced performance profile (low latency desktop)
  2 = Default performance profile
  3 = Tuned performance profile (high latency desktop)
```

```
$ ./hashcat.bin -m 0 -a 3 -w 1 example0.hash ?a?a?a?a?a?a?a?a?a
```

However, be aware that this also will decrease your speed.

Is the 64 bit version faster than the 32 bit version?

- For hashcat legacy, yes. That is because the hashes are computed on CPU (and your CPU has most likely a 64 bit architecture).
- For hashcat (by only using your GPUs), no. You can however allocate more memory and that might help with other problems.

Generally you should use the 64 bit versions as these are the ones the developer use, too.

What is it that you call "GPU power"?

The GPU power is simply the amount of base-words (per GPU) which are computed in parallel per kernel invocation. Basically, it's just a number: $S * T * N * V$

- S = Shader. You can see the number of shaders of your GPU on startup. For example, 32 on hd7970
- T = 64 for AMD, 256 for NV
- N = `-n` value, typically 256 if you use `-w 3`
- V = vector-size, depends on hash-type. Typically 1, 2 or 4

How to create more work for full speed?

(short [URL](https://hashcat.net/faq/morework): <https://hashcat.net/faq/morework>)

This is a really important topic when working with Hashcat. Let me explain how Hashcat works internally, and why this is so important to understand.

GPUs are not magic superfast compute devices that are thousands of times faster than CPUs – actually, GPUs are quite slow and dumb compared to CPUs. If they weren't, we wouldn't even use CPUs anymore; CPUs would simply be replaced with GPU architectures. What makes GPUs fast is the fact that there are thousands of slow, dumb cores (shaders.) This means that in order to make full use of a GPU, we have to parallelize the workload so that each of those slow, dumb cores have enough work to do. Password cracking is what is known as an "embarrassingly parallel problem" so it is easy to parallelize, but we still have to structure the attack (both internally and externally) to make it amenable to acceleration.

For most hash algorithms (with the exception of very slow hash algorithms), it is not sufficient to simply send the GPU a list of password candidates to hash. Generating candidates on the host computer and transferring them to the GPU for hashing is an order of magnitude slower than just hashing on the host directly, due to PCI-e bandwidth and host-device transfer latency (the PCI-e copy process takes longer than the actual hashing process.) To solve this problem, we need some sort of workload amplifier to ensure there's enough work available for our GPUs. In the case of password cracking, generating password candidates on the GPU provides precisely the sort of amplification we need. In Hashcat, we accomplish this by splitting attacks up into two loops: a "base loop", and a "mod(ifier) loop." The base loop is executed on the host computer and contains the initial password candidates (the "base words.") The mod loop is executed on the GPU, and generates the final password candidates from the base words on the GPU directly. The mod loop is our amplifier – this is the source of our GPU acceleration.

What happens in the mod loop depends on the attack mode. For brute force, a portion of the mask is calculated in the base loop, while the remaining portion of the mask is calculated in the mod loop. For straight mode, words from the wordlist comprise the base loop, while rules are applied in the mod loop (the on-GPU rule engine that executes in the mod loop is our amplifier.) For hybrid modes, words from the wordlist comprise the base loop, while the brute force mask is processed in the mod loop (generating each mask and appending it to base words is our amplifier.)

Without the amplifier, there is no GPU acceleration for fast hashes. If the base or mod loop keyspace is too small, you will not get full GPU acceleration. So the trick is providing enough work for full GPU acceleration, while not providing too much work that the job will never

complete. For straight mode against fast hashes, your wordlist should have at least 10 million words and you should supply at least 1000 rules.

Now, we mentioned above that this advice is for most hash algorithms, with the exception of very slow hash algorithms. Slow hash algorithms use some variety of compute-hardening techniques to make the hash computation more resource-intensive and more time-consuming. For slow hash algorithms, we do not need (nor oftentimes do we want) an amplifier to keep the GPU busy, as the GPU will be busy enough with computing the hashes. Using attacks without amplifiers often provide the best efficiency.

Because we are very limited in the number of guesses we can make with slow hashes, you're often working with very small, highly targeted wordlists. However, sometimes this can have an inverse effect and result in a wordlist being too small to create enough parallelism for the GPU. There are two solutions for this:

- Use rules, but not as an amplifier. Basically this means you feed Hashcat base words through a pipe:

```
$ ./hashcat.bin --stdout wordlist.txt -r rules/best64.rule | ./hashcat.bin -m 2500 test.hccapx
```

- Use princeprocessor. Same processes over a pipe:

```
$ ./pp64.bin wordlist.txt | ./hashcat.bin -m 2500 test.hccapx
```

- Sometimes it can make sense to use maskprocessor. Note this should be used only for very small keyspaces!

```
$ ./mp64.bin test?d?d?d?d?d?d | ./hashcat.bin -m 2500 test.hccapx
```

Note: pipes work in Windows the same as they do in Linux.

Those attack modes are usually already built into Hashcat, so why should we use a pipe? The reason is, as explained above, masks are split in half internally in Hashcat, with one half being processed in the base loop, and the other half processed in the mod loop, in order to make use of the amplification technique. But this reduces the number of base words, and for small keyspaces, reduces our parallelism, thus resulting in reduced performance.

Is piping a wordlist slower than reading from file?

No, piping is usually equally fast.

However, most candidate generators are not fast enough for hashcat. For fast hashes such as MD5, it is crucial to expand the candidates on the GPU with rules or masks in order to achieve full acceleration. However be aware that different rulesets are not producing constant speeds. Especially big rulesets can lead to a significant speed decrease. The increase from using rules as amplifier can therefor cancel itself out depending how complicated the rules are.

Why is my attack so slow?

- To find out about what maximum speeds you can expect from your system, run a benchmark [How can I perform a benchmark?](#) Note: Benchmarks are a "best case" scenario, i.e. single hash brute force. Real-world speed can vary depending on the number of hashes and attack mode.
- In most of the cases of "slow speeds" you simply did not create enough work for hashcat. Read [How to create more work for full speed?](#)
- You can add more pressure on the GPU using the -w 3 parameter. Note: this will cause your desktop to lag because the GPU is so busy it can not compute desktop changes like mouse movement.
- Your GPUs are overheating. If this happens (typically around 90c) the GPU bios automatically downclocks the GPU = slower speed
- The more hashes are in your hashlist, the slower the speed gets. The biggest difference is between one or more hashes because for single hashes hashcat can use special optimizations which only can be used when cracking just a single hash
- Some hashes are designed to run slow, like bcrypt, scrypt or bitcoin wallet. Deal with it.

Why does hashcat says it has only 2% GPU utilization?

Not having 100% GPU utilization is usually an indicator for a too-small keyspace (meaning, not enough work to be done).

Read [How to create more work for full speed?](#)

How is it possible that hashcat does not utilize all GPUs?

If the number of base-words is so small that it is smaller than the GPU power of a GPU, then there is simply no work left that a second, or a third, or a fourth GPU could handle.

Read [How to create more work for full speed?](#)

Why does hashcat sometimes get very slow at the end of an attack?

First we need to define "what is the end of an attack". oclHashcat defines this for the following case:

If the number of base-words is less than the sum of all GPU power values of all GPU. Read [What is it that you call "GPU power"?](#)

This happens when you see this message:

```
INFO: approaching final keyspace, workload adjusted
```

If this happens, hashcat tries to balance the remaining base-words to all GPU. To do this, it divides the remaining base-words with the sum of all GPU power of all GPUs which will be a number greater than 0 but less than 1. It then multiplies each GPU power count with this number. This way each GPU gets the same percentage of reduction of parallel workload assigned, resulting in slower speed.

Note that if you have GPUs of different speed it can occur that some GPU finish sooner than others, leading to a situation when some GPU end up in 0 H/s.

Why is hashcat taking so long to report progress?

(short [URL](#): <https://hashcat.net/faq/slowprogress>)

This is a problem related to (a) how GPU parallelization works in general in combination with (b) an algorithm with a very iteration count.

When it comes to modern hashing algorithms they are typically designed in a way that they are not parallelizable and that the calculation has to be done in serial. You can not start computing iteration 2 if you have not computed iteration 1 before. They depend on each other. This means for slow algorithms like 7-Zip (if we want to make use of the parallelization power of a gpu) we have to place a single password candidate on a single shader (which a gpu has many) and compute the entire hash on a it. This can take a very long time, depending on the iteration count of the algorithm. We're talking about times up to a minute here for a single hash computation. But what we got for doing it is that we're able to run a few 100k at the same time and make use of the parallelization power of the gpu in that way. That's why it takes so long for hashcat to report any progress, because it actually takes that long to compute a single hash with a high iteration count.

In the past hashcat did not report any speed for such extreme cases, resulting in hashing speed of 0H/s. Some users may remember such cases and wondering "why isn't it doing anything". From a technical perspective nothing changed. In the past and also today the GPU just need that much time. The only difference is in newer hashcat versions is that it creates an extrapolation based on the current progress of the iterations. For example it knows that the shader processed like 10000 of 10000000 iterations in X time and therefore it can tell how much it will (eventually) take to process the full iteration count and recomputes this into a more or less valid hashing speed, which is in that case not the real one. This is what you are shown in the speed counter.

But that's the theoretical part. When it comes to GPGPU it's actually not enough to feed as many password candidates to the GPU as it has shaders. There's a lot of context switching going on the GPU which we have no control over. To keep all the shaders busy for the entire time we

have to feed it with password candidates many times the number of shaders.

(See [this forum post](#) for an illustration with examples)

Special features

Can I restore a hashcat session?

The command line switch you are looking for is `--restore`.

The only parameters allowed when restoring a session are:

- `--restore` (required): tell hashcat that it should restore a previous session
- `--session` (optional): specify the session name of the previous session that hashcat should restore

Note: if you did use `--session` when starting the cracking job, you also need to use `--session` with the same session name to restore it.

Further parameters and switches are currently not allowed, e.g. you can't simply add `-w 3` when restoring (i.e. `--restore -w 3`) because it will be ignored. If you really know what you are doing and want to change some parameters in your `.restore` file, you might want to use some third-party tool like [analyze_hc_restore](#)

Also see [restore](#) for more details about the `.restore` file format.

Can I restart hashcat on a different PC or is it possible to add a new GPU to my system?

Yes! All you need to ensure is that no files have been modified.

The most important file here is the `.restore` file (the file name depends on the session name used, see `--session` parameter, so it is `$session.restore`). You need to copy at least the original hash list and the `.restore` file to the new computer.

Therefore, if you move to a different PC make sure all the paths are the same and all files exist.

To get more information about which files we mean you can use this utility to find out: https://github.com/philsmd/analyze_hc_restore

How can I distribute the work on different computers / nodes?

If you want to make use of multiple computers on a network, you can use a distributed wrapper.

There are some free tools:

1. Hashtopolis (even works over internet connections), original code from Hashtopus, renamed (was Hashtopussy)
 - <https://github.com/s3inlc/hashtopolis> (original version: <https://github.com/curlyboi/hashtopus>)
 - <https://github.com/s3inlc/hashtopolis-agent> (original version: <https://github.com/curlyboi/hashtopus-agent>)
 - <https://github.com/s3inlc/hashtopolis-agent-python>
 - <https://github.com/s3inlc/hashtopolis-agent-csharp>
2. Disthc
 - <https://github.com/unix-ninja/disthc>

There are also some proprietary commercial solutions:

1. Hashstack
 - <https://sagitta.pw/software/>

We do neither develop nor maintain, nor directly support, any of these third-party tools. Please contact the authors of these tools directly if you have any questions.

I read somewhere to use VCL for distributed cracking, is this still a thing?

VCL was one of the first tries for distributed hashcat, but there are better alternatives today.

See this: [How can I distribute the work on different computers / nodes?](#)

Can hashcat send an email once a hash has been found?

No. In order to achieve this, you will need to wrap your hashcat attack in a script that sends an email when hashcat is finished.

Why is my script unable to communicate with the hashcat / hashcat legacy prompt?

The reason behind is that hashcat and hashcat legacy have this prompt:

```
[s]tatus [p]ause [r]esume [b]ypass [q]uit =>
```

The problem with Linux and Windows in this case is that if a user would press "s" it would be buffered until the users also hits enter.

To avoid this, we have to put hashcat into the canonical mode and set the buffersize to 1.

You can still communicate with the process, but you have to spawn your own PTY before you call hashcat to do so.

I got a hash cracked on a different computer, can I tell hashcat about that while it is running?

The answer is yes - but before we explain how to do it, let's answer the question of why you want to do it. The answer is simple, especially when you're cracking salted hashes.

Imagine that you have a large hashlist with 100 salts. This will reduce your guessing speed by a factor of 100. Once all hashes bound to a given salt are cracked, hashcat notices this and skips over that specific salt. This immediately increases the overall performance, because now the guessing speed is only divided by 99. If you crack another salt, the speed is divided by 98, and so on. That's why it's useful to tell hashcat about cracked hashes while it's still running.

You may have already noticed that when you start hashcat, a 'hashcat.outfiles' directory is automatically created (more correctly the `*.outfiles` directory depends on the session name, see `--session`, so it is `$session.outfiles`).

This directory can be used to tell hashcat that a specific hash was cracked on a different computer/node or with another cracker (such as hashcat-legacy). The expected file format is not just plain (which sometimes confuses people), but instead the full hash[:salt]:plain.

For instance, you could simply output the cracks from hashcat-legacy (with the `--outfile` option) to the `*.outfiles` directory, and hashcat will notice this immediately (depending on `--outfile-check-timer`).

The parameters that you can use to modify the default settings/behavior are:

```
--outfile-check-dir=FOLDER    Specify the outfile directory which should be monitored, default is $session.outfiles
--outfile-check-timer=NUM      Seconds between outfile checks
```

hashcat will continuously check this directory for new cracks (and modified/new files). The synchronization between the computers is open for almost any implementation. Most commonly, this will be an NFS export or CIFS share. But in theory it could be also synced via something like rsync, etc.

Note: some users **confuse** the induction (--induction-dir) and loopback (--loopback) feature with the feature mentioned above, but they are (very) different:

- outfile check: as described above in detail, this is used when you already exactly know some hash[:salt]:plain combinations, because a subset or all of the hash(es) were already cracked on other computer/nodes or with other crackers
- induction: in the induction directory you can add some (new) plains that hashcat should load/use after the current dictionary is finished
- loopback: re-use the plains/passwords that did crack a hash, e.g. apply some rules - after the first run - to the modified and matching plains. This kind of looping will only stop if no more plains match.

How do I extract the hashes from Office (Word, Excel, etc.) documents?

Please use the python script [office2hashcat.py](#) to extract the required information from an office file.

After you have extracted the "hashes" with the script linked above, you need to either know the office version number or compare the hashes directly with the [example_hashes](#). Depending on the version number/signature of the hashes you select the corresponding hash mode and start hashcat with the -m value you got.

How do I extract the hashes from PDF documents?

You should use the [pdf2hashcat.py](#) tool to extract the required information of the .pdf files. The output "hashes" could have different signatures depending on the pdf version. For some example hashes see: [example_hashes](#) (-m 10400, -m 10500, -m 10600 or -m 10700).

pdf "hashes" with different hash types (-m values) need to be cracked separately, i.e. you need to have different cracking jobs for each hash type and specify the correct -m value. But if several hashes were generated by the same PDF software version, they can be cracked together and the hash file would look like any other multi-hash file (one hash per line).

How do I extract the hashes from TrueCrypt volumes?

In order to crack TrueCrypt volumes, you will need to feed hashcat with the correct binary data file. Where this data lives depends on the type of volume you are dealing with.

The rules are as follows:

1. for a TrueCrypt boot volume (i.e. the computer starts with the TrueCrypt Boot Loader) you need to extract 512 bytes starting with offset 31744 (62 * 512 bytes). This is true for TrueCrypt 7.0 or later. For TrueCrypt versions before 7.0 there might be different offsets.

Explanation for this is that the volume header (which stores the hash info) is located at the last sector of the first track of the system drive. Since a track is usually 63 sectors long (1 sector is 512 bytes), the volume header is at sector 63 - 1 (62).

2. if TrueCrypt uses a hidden partition or volume, you need to skip the first 64K bytes (65536) and extract the next 512 bytes.

```
dd if=hashcat_ripemd160_AES_hidden.raw of=hashcat_ripemd160_AES_hidden.tc bs=1 skip=65536 count=512
```

3. in all other cases (files, non-booting partitions) you need the first 512 Bytes of the file or partition.

You can extract the binary data from the raw disk, for example, with the Unix utility **dd** (e.g. use a block size of 512 and a count of 1).

You need to save this hash data into a file and simply use it as your hashlist with hashcat.

The hashcat wiki lists some TrueCrypt [example_hashes](#) (e.g. -m 6211, -m 6221, -m 6231 or -m 6241 depending on the exact TrueCrypt settings that were used when setting up the TrueCrypt volume). If you want to test/crack those example "hashes", as always, use the password "hashcat" (without quotes).

The same procedure should also work for **VeraCrypt** volumes (but you need to adapt the hash mode to -m 137XY - see the --help output for all the supported hash modes for VeraCrypt and the correct values for X and Y).

How do I extract the hashes from VeraCrypt volumes?

The procedure to extract the important information from data encrypted with VeraCrypt follows the same steps/rules as for TrueCrypt: see [How do I extract the hashes from TrueCrypt volumes?](#)

It's important that you do not forget to adapt the hash mode (-m). For all supported hash modes for data encrypted with VeraCrypt, please have a glance at the --help output.

How can I crack passwords from htpasswd?

The format of Apache htpasswd password files does support several hashing algorithms, for instance Apache MD5 ("\$apr1"), raw sha1 ("{\$SHA}"), DEScript, etc

Depending on the signature, you need to select the correct hash type (-m value). See [example_hashes](#) for some examples.

The format of htpasswd lines is:

```
user:hash
```

You do not need to remove the username at all, you can just simply use the --username switch.

An example of the (still) most-widely used format found is -m 1500 = DEScript:

```
admin:48c/R8JAv757A
```

(the password here is "hashcat")

How can I crack SL3?

Start by going somewhere else. We don't care where you go from there.

The story behind this: <https://hashcat.net/forum/thread-1887.html>

How can I crack multiple WPA handshakes at once?

The [.hccapx file format](#) allows to have multiple networks within the same .hccapx file.

This means that a single .hccapx file can also consist of multiple individual .hccapx structures concatenated one to each other.

For Linux / OSX systems you should use a command similar to this one:

```
$ cat single_hccapxs/*.hccapx > all_in_one/multi.hccapx
```

and for windows systems this:

```
$ copy /b single_hccapxs\*.hccapx all_in_one\multi.hccapx
```

The file multi.hccapx (in this particular case) would then consist of all the networks within the single_hccapxs/ folder.

[cap2hccapx](#) is also able to convert a .cap file with several network captures to a single ("multi") .hccapx file.

Note: the concatenated networks do not need to origin from the same .cap capture file, there is no such limitation on where the captures should come from, but they must be valid/complete captures of course.

There are 2 possible reasons why some password candidates are being rejected:

- the algorithm itself has some limitations: minimum and/or maximum password length
- hashcat has some upper limits of password length depending on the attack mode (-a value)

This can also be seen in the status screen:

In this particular case 1 out of 4 password candidates (25%) were rejected by hashcat.

```
WARNING: skipping mask '?1?1?1?1?1?1?1?1' because it is larger than the maximum password length
```

This also brings up an important point, if we apply some rules (-j/-k/-r) or combine several words (-a 1), it is not always possible to reject password candidates immediately by the host (CPU). Therefore, it is possible that the password candidates or words already "reached" the GPU (they were copied to the GPU) but can't and won't be rejected by the host and be counted as rejected, since only the GPU can decide if they should be rejected. This is because hashcat uses a rule engine directly on GPU and can combine plains on your graphics cards too.

What is the maximum supported password length?

The maximum supported password length is 256 characters/bytes. This is only true for pure kernels, for optimized kernels (i.e. if you are using the -O option or if there are only optimized kernels for the hash type you are using) the maximum password length is lower and follows the rules mentioned in [What is the maximum supported password length for optimized kernels?](#)

The maximum supported salt length is 256 characters/bytes. This is only true for pure kernels, for optimized kernels (i.e. if you are using the -O option or if there are only optimized kernels for the hash type you are using) the maximum salt length is lower and follows the rules mentioned in [What is the maximum supported salt length for optimized kernels?](#)

There's no easy or general answer. Thing is, it depends on many factors.

- Zero-based optimizations: Many algorithms can be optimized based on the fact that zero values in arithmetic or logic operations do not change the input value. With a password limit of less than 16 characters, it was guaranteed that values for positions 16-63 were zero, allowing us to omit dozens of operations from each step. These optimizations can no longer be used. See Passwords13 presentation: <http://hashcat.net/p13/js-ocohaaaaa.pdf>
- Register pressure: If a password is 15 characters or less, we only need four 32-bit registers to hold it. But as we increase the length, more registers are required. This can slow down the entire process if more registers are required than accessible, as the compiler has to swap out data to global memory. Supporting passwords longer than 15 characters increases register pressure.
- Wordlist caching: hashcat handles wordlist data in a very unique way. The words are not simply pushed to GPU from start to finish; they are first sorted by length. This is done because many algorithms can be further optimized when all of the input data is the same length. This required a lot of host memory. Depending on the number of GPUs we have and the specified -n value, oclHashcat-plus easily allocated 16GB of host memory or more. This buffer would have been increased 4x since we wanted to increase from a maximum length of 16 to a maximum length of 64. In other words, our host system would request 64GB of RAM!
- Branching: Many (nearly all) hash algorithm support any password length as input. To do this, the password is split into blocks of a specific size, for example 64 byte for MD5. The hash computes the first 64 byte and then uses the resulting digest as initialization value for the next 64 byte, and so on... To be able to do this, it has to branch (means using if() statement) and GPU's hate branching

- Limitation from the hash itself:

- 1500: 8
- 3000: 7
- 9710: 5
- 9810: 5
- 10410: 5
- For slow hashes:
 - 400: 40
 - 500: 16
 - 1600: 16
 - 1800: 16
 - 2100: 16
 - 5200: 24

- 5300: 16
- 5800: 16
- 6300: 16
- 7400: 16
- 7900: 48
- 8500: 8
- 8600: 16
- 10300: 40
- 10500: 40
- 10700: 16
- 11300: 40
- For fast hashes, the important factor is the attack mode:
 - attack-mode 0, the maximum length is 31
 - attack-mode 1, the maximum size of the words of both dictionaries is 31
 - attack-mode 6 and 7, the maximum size of the words of the dictionary is 51

Just to make this clear: We can crack passwords up to length 55, but in case we're doing a combinator attack, the words from both dictionaries can not be longer than 31 characters. But if the word from the left dictionary has the length 24 and the word from the right dictionary is 28, it will be cracked, because together they have length 52.

Also note that algorithms based on unicode, from plaintext view, only support a maximum of 27. This is because unicode uses two bytes per character, making it $27 * 2 = 54$.

What is the maximum supported salt length for optimized kernels?

The maximum supported salt-length, in general, for the generic hash-types is 31.

If you came here you are probably looking for the maximum salt-length for the generic hash-types like MD5(\$pass.\$salt) or HMAC-SHA256 (key = \$salt). This is because of all the other special (named) hash-type like Drupal7 we set the salt length according to the hash-type specification of the application using it. This means you wouldn't ask for it because you will not run into a problem with it.

What you cannot do is increase this limit. But you can request a new specific hash-type to be added that has different default limits. This makes sense if the application is somehow prominent enough to be added as a special named hash-type. The correct way of asking for a new hash-type to be added is described here: [I want to request some new algorithms or features, how can I accomplish this?](#)

I do not want hashcat to reject words from my wordlist if they are too long, can it truncate them instead?

That's indeed possible and very simple. For example, if you're going to crack DEScript hashes, they have a maximum length of 8. If you run a typical wordlist on it, for example "rockyou.txt" there are many password of length 9 and more.

That means hashcat will reject them.

There's a simple way to avoid this. If you truncate all words from the wordlist to length 8 it will not skip them. This can be done on-the-fly using the -j rule.

```
'8
```

The ' rule means truncate. This has some negative effects, too. For example imagine your wordlist contains something like this:

```
password1
password1234
password1337
```

Truncating them at position 8 means that all of them will result in simply "password". This will create unnecessary double checks.

How can I perform a combinator attack with three wordlists (triple combinator, 3-way combinator)?

 hashcat-utils ships with an command line utility called combinator3. This tool allows one to specify 3 (different or identical) wordlists as command line parameters and it will combine each word within the first wordlist, with each word from the second one and each word from the third wordlist.

```
$ ./combinator3.bin dict1.txt dict2.txt dict3.txt
```

Note: the total number of resulting password candidates will be determined by words_in_dict1 * words_in_dict2 * word_in_dict3. From this formula it should be clear, that this total number of combinations and resulting words will be very high depending on the number of lines of the 3 files.

In some (rare) cases it could make more sense to use -a 1 (combinator attack) with -j / -k rules to prepend/append a static plain or pipe combinator2 to hashcat and apply some rules with the -r argument. It depends from case to case which method is faster and/or easier.

Can I load multiple hashlists at once?

This is not supported.

If all the hashes in your hashlists are of the same hash-type it is safe to copy them all into a single hashlist.

If they are not of the same hash-type you can still copy them all into a single hashlist but note that if you use the --remove parameter then all valid hashes that could be successfully parsed with the hash type specified are dropped and only the "matching" uncracked hashes will remain in the list.

I want to skip password candidates that have repeating characters. Is this possible?

Well, it's not supported built-in. But maskprocessor supports this feature through it's -q option. You could simply pipe the output of maskprocessor to hashcat (or use fifos with hashcat legacy).

That looks like the following:

```
$ ./mp64.bin -q 3 ?d?d?d?d?d?d?d | ./hashcat.bin -m 2500 test.hccapx ...
```

There is also another feature supported by maskprocessor which deals with maximum number of total occurrences of a single character (i.e. the characters do not need to be directly attached to each other to be refused):

```
$ ./mp64.bin -r 3 ?d?d?d?d?d?d?d | ./hashcat.bin -m 2500 test.hccapx ...
```

This prevents that maskprocessor will output password candidates with more than 2 identical digits (in this particular case) within the whole line.

So please do not confuse -q (limit on consecutive identical characters) with -r (limit on identical characters within the whole password candidate/string).

Note that using a pipe works on windows the same as on linux. Therefore the solution above does work on windows, too.

LM consists of two hashes but are split for use with hashcat. How to get them back into one piece?

This question is kind of outdated. Current hashcat versions have native support for this.

- If the hashes are 16 chars long, --show will show the hash (part¹) and plain if cracked, and --left will show the hash parts which were not cracked

ELSE if the hash is 32 chars long ("both parts" of the hash in one line):

- The output will contain [notfound] within the password part of the output that was not cracked, e.g. [notfound]abc means that the second part was cracked while the first one was not, hashcat[notfound] means that the second part was not found
- hashcat checks if the 2nd part is the empty string hash (weak hash) and if this is the case it will be automatically marked as cracked (regardless if in .potfile or not)
- If both parts were found (2nd part could be weak hash too), there will be no [notfound] in the output line
- --left will only show the **parts** that were not cracked or the whole hash (2 parts) if both parts were not found

¹ the reasons behind this "parts" is that LM hashes can consist of a password length of maximum 14 characters. But the important detail is that the hashes can be splitted into 2 individual hashes (which hashcat of course does to optimize the attack, i.e. hashcat could also crack the easier part of the 14 char password separately, and the user could attack the second part by doing educated guesses with the information given), each of the parts have a maximum password length of 7

Btw. also the --username switch and the --outfile-format command line argument work with this --show feature for LM hashes

IOW, to make use of it, you need to use the --show parameter. To understand how to use --show correctly, refer to this article:

[How can I show previously cracked passwords, and output them in a specific format \(e.g. email:password\)?](#)

Here's an example:

```
$ cat hash
ec97b64dce94d891299bd128c1101fd6
```

```
$ ./hashcat.bin -m 3000 -a 3 -w 3 -o cracked.txt --quiet hash ?u?u?u?u?u?u
```

Here's the "normal" outfile, one hash per line:

```
$ cat cracked.txt
ec97b64dce94d891:AMAZING
299bd128c1101fd6:HASHCAT
```

But now let's put them back into one piece:

```
$ ./hashcat.bin -m 3000 --show --quiet hash
ec97b64dce94d891299bd128c1101fd6:AMAZINGHASHCAT
```

PS: You can also use -o to write to an outfile

Hardware

How can I make sure that my GPU is supported?

- For NVidia GPUs see:  supported NVidia (cuda) GPUs
- For AMD GPUs see:  supported AMD GPUs

Please also always double-check that you are using the  recommended driver.

There are some GPU's that were supported in the past but they are no longer supported with recent drivers.

- For NVidia GPU's that is all GPU that base on ShaderModel < version 2, ex. GTX280
- For AMD GPU's that is all GPU from the hd4xxx series

What is the best GPU for hashcat for "total speed", "speed by watt" and "speed by price"?

Due to the nature of this subject changing continuously there is no definitive answer. In regards to the behaviour of "speed by watt" your best choice have been the high end dual GPU cards. For more details on "total speed" and "speed by price" see [Is there a general benchmark table for all GPU?](#)

Is there a general benchmark table for all GPU?

ATM, there is no such table. There have been projects trying to do this:

-  http://thepasswordproject.com/oclhashcat_benchmarking

... but they're all outdated for todays versions

As for today, this list seems to be the most complete one:

-  https://hashcat.net/misc/p130_img/changes_v130.png

... but even this one is outdated.

Does the PCI-Express speed have any influence on cracking speed?

Actually, yes. But in reality, no. If you go for pure Brute-Force it has absolutely no effect.

If you go for wordlist based attacks, the only difference is how long it takes for your host system to copy new wordlist data to your GPU. While it copies the data, the GPU is busy and can not compute. For example with PCI-E v2.0 x1 we can do 4 Gbit/s and with x16 we can do 64 Gbit/s. This sounds like alot, but it's not. First this number is bits not bytes and more important this is the speed per second. We also measure guessing speed in X per second, that means if we take a full second to transfer a specific amount of data the guessing speed is 0. That means, to not influence with guessing speed we need to be able to transfer our wordlist data in less than a percent of a second.

Since hashcat has to pad wordlist data to achieve maximum performance while computing new hashes one can expect that every word, whatever its real length is, takes around 256 byte + 16 byte overhead for other data. Now imagine you have a hd7970 which comes with 32 compute units and an ideal thread count of 64 we end up in: $32 * 16 * N\text{-parameter} * 272$. If you use -w 3 and MD5, the N-parameter is set to 256. Finally, this is 272 Mbit. While we have 4 Gbit/s with x1 this means that our copy buffer takes 4096/272 percent of a second to finish. This is around 15% percent. So for x1 we end up with a guess speed reduction of 15% while it's only 1% for x16.

However, don't worry about it. With hashcat we workaround this problem by using an amplifier. For example if we add a rulset of 5000 rules we reuse the same wordlist segment 5000 times, effectively reducing the copy-buffer overhead of 15% with x1 to 15%/5000 which is nearly nothing. So if your amplifier is big enough, don't worry about PCI-E speed.

Should I attach Crossfire (AMD) / SLI (NVidia) bridges?

If possible, try to avoid it. These bridges are meant to be used for applications (games) that have no explicit support for multiple GPUs. Therefore the driver fakes a single GPU and tries to balance the workload across the real existing GPU. This, of course, creates a lot of overhead that we don't want and don't need, since hashcat natively knows how to deal with multiple GPUs.

My GPU overheats sometimes. Can hashcat automatically pause in case of overheating or with a script?

This is a bad idea and therefore hashcat doesn't support it. What makes more sense is to either use the command line parameter `-w 1` to enable the "Reduced performance profile" or to play around with the `--gpu-loops` (short `-u`) and/or `--gpu-accel` (short `-n`) parameters directly.

Another thing you could test is to manually put your GPU fan speeds (if you are not using water cooling) to 100% and use `--gpu-temp-disable` (but this is not recommended in general).

The best way to fix the problem is to invest some time investigating in how to improve the cooling and if you finally have found some ways to do so, invest some money in better cooling hardware (more/better high pressure fans) or improved air flow.

Why does the temperature reported on the status screen not match the temperature in Afterburner?

Afterburner has not to rely on OpenCL and therefore can not fail matching OpenCL and ADL devices.

See here for a detailed description: [What does the ADL_Overdrive5_Temperature_Get\(\): -1 error mean?](#)

How many GPU can hashcat handle?

In theory there's no limit for hashcat.

However there are limits to the runtime hashcat rely on. For example with OpenCL or CUDA. As far as we know there are no hardcoded limits to OpenCL or CUDA, but then there's X11 in the middle (Linux) and then comes the driver area. For AMD, at least the drivers are known to be limited to a maximum of 8 GPU. Which means 4 physical graphic cards if they are dual-gpu.

Generally it's a better idea to use multiple computers with only a few GPUs per system. For example, if you use a distributed solution ([How can I distribute the work on different computers / nodes?](#)) then there's really no limit to how many systems you attach to your cluster.

I was doing bitcoin and litecoin mining, can I use my FPGA or ASIC for hashcat?

FPGA are sub-optimal for advanced password cracking in a few key ways. They are best for brute forcing single hash of a single algorithm (like bitcoin). They do not provide the flexibility needed for multiple attack modes, multiple hashes, or multiple algorithms. Too much would have to be done on the host system.

The problem with ASIC is that they are, by definition, application-specific. Bitcoin ASIC will only work for bitcoin, and nothing else. Well, you could attempt to use them for password cracking, but you would only be able to crack passwords that were exactly 80 characters long and hashed as double SHA256. So, virtually worthless for anything but bitcoin.

By the same token, building ASIC specifically for password cracking would be a huge waste of time and money. And to make an ASIC that was flexible enough to handle multiple hashes, multiple algorithms, and multiple attack modes, you'd essentially just end up with a GPU. They really are the sweet spot. cheap, fast, flexible, easy to program.

Can I use Intel GPUs, they support OpenCL?

Intel has great OpenCL support on Windows, but no support on Linux. Intel's OpenCL SDK for Linux supports only CPU.

Since hashcat is programmed on Linux (and afterwards cross-compiled for windows) there's no chance yet to getting this to work.

GPUs are not magic go-fast devices. The microarchitecture and ISA have to be well-suited for the task at hand. As it stands, Intel GPUs have very minuscule raw compute power, and their ISA is not optimal for password cracking. Most modern-day CPUs with XOP or AVX2 support will be faster than an Intel GPU.

Why does hashcat show the wrong GPU memory when I start it?

That is because the OpenCL runtime does not have access to the full available GPU memory. There are some special limitations. Also with the latest driver version it seems the runtime only sees the remaining allocatable memory only. This creates even more confusion to the user. However it's nothing that hashcat has any influence in.

For NVIDIA specifically, a hard limit of 25% of GPU memory being made available to a single OpenCL allocation 🐞 has been observed in the wild. As of 2017-07, there is an open RFE with NVIDIA on this issue.

Common errors

I created a hash with "echo test | md5sum" but hashcat fails to crack it

The error in here is that the echo command adds a newline character (`\n`). To avoid this you need to use the `-n` parameter.

```
$ echo test | md5sum
d8e8fca2dc0f896fd7cb4cb0031ba249 -
```

```
$ echo -n test | md5sum
098f6bcd4621d373cade4e832627b4f6 -
```

Line-length exception, wait what?

This error may have different reasons, which will all be mentioned shortly, but it means in general that the hash file (or it could be also the hash specified on command line in case of hashcat) could not be parsed successfully and therefore the hash could not be loaded.

There are 2 versions of an error message that could be shown:

case1 (file does exist but could not be successfully parsed):

```
./hashcat.bin -m 0 md5_hashes.txt dict.txt
-snip-

WARNING: Hashfile 'md5_hashes.txt' in line 1 (010203invalidhash): Line-length exception
```

case2 (file does not exist but hashcat tries to parse the file name as a hash specified on command-line):

```
./hashcat.bin -m 0 non_existing_file.txt dict.txt
-snip-

WARNING: Hash 'non_existing_file.txt': Line-length exception
```

Now that we know the different type of error messages that will be shown, we will investigate also the most common reasons for these error messages:

1. incorrect hash type: make sure that you always specify the correct hash type (--hash-type command line option or short -m)
2. whitespace: spaces, tabs etc around the hash (in most cases at the end of a line within the hash file) won't be accepted and therefore must be removed.
The reason why hashcat does not automatically remove the white spaces for the user (if it detects a line-length exception) is that (silently) removing parts of the characters in the hash line (hash or salt) is in most of the cases not what the user expects hashcat/hashcat legacy to do and in some cases (for instance salted hashes and the salt contains a space at the end) even something that could lead to problems and unexpected results.
3. username: if the lines within the hash file are formatted as user:hash (and not hash alone), you need to specify the --username switch
4. hash file was not found (hashcat only): if the hash file specified on the command line was not found, hashcat tries to load the command line option as a hash directly (case2 above). If this is the case and you didn't want to specify a hash directly on command line, please double-check that the path to the hash file is correct.

Depending on the type of error, you will either see the variants mentioned in case1 or case2 of the error message. For case2 you need to troubleshoot that the file either exists (if you indeed want to specify a hash file) or that the format of the hash specified on the command line is correct.

To make sure that the hash follows the hash formats, visit this [example hashes](#) wiki page.

When I use --remove with hashcat legacy the hash sometimes remains even if it was cracked

That is because hashcat legacy does not sort out double hashes of the input hashlist. If you have multiple times the same hash in your hashlist it will always crack only the first. This means, even if you use --remove, it's possible that you end up with a hashlist that contains a cracked hash. To avoid such problems make sure you remove duplicate hashes from your hashlist before you run it with hashcat legacy.

```
$ LC_ALL=C sort -u hashlist.txt | sponge hashlist.txt
```

Since hashcat automatically removes such duplicate hashes on startup you don't have to worry about this.

Compatibility

Why does hashcat not work with my Kali operating system?

In theory, it should. The only problems we can imagine are that either Kali is simply using an invalid driver or that you did not download hashcat directly from <https://hashcat.net/hashcat> and that the hashcat version used is not up to date.

In the past, there was a problem where Kali still used a very old glibc that was incompatible with the one from Ubuntu. When we compiled new hashcat or hashcat-legacy binaries, the compiler used the glibc from the host system. To work around the problem, we switched to a hashcat-legacy-specific toolchain, which uses an older glibc that is compatible with the one used in Kali. So this specific problem should not exist anymore.

Can I use JtR rules with hashcat?

Most of them, yes. There are some functions that are not supported by hashcat / hashcat legacy. The [rule syntax documentation](#) shows which rules are compatible with JtR.

However, in case you use such an unsupported rule, both hashcat and hashcat legacy simply skip over them and gives you a warning, but they are not applied. This means you can simply use them and the ones that are fully compatible are applied.

The preprocessor "[" and "]" from JtR is not supported. Please use maskprocessor to generate those rules.

Which wordlist are recommended for WPA cracking?

This is a typical error. There can't be specific wordlist for specific hash-type targets. One can argue that WPA does not allow words < length 8 but for this case hashcat has a built-in filter. That means, hashcat knows all the different minimum and maximum limits of a specific hash-type and filters non-matching words from your wordlist on-the-fly. Don't worry about such cases!

Organizational

I want to request some new algorithms or features, how can I accomplish this?

The preferred method is to use [github issues](#).

Please make sure that the feature/problem was not already reported by using the search function.

A short reminder about the important information a github issue needs:

- Problem/bug:
 - detailed description what exactly does not work, e.g. give a clear description of the problem
 - description when exactly the problem occurs (and when it doesn't)
 - full command (but reduced to the bare minimum - of parameters etc - needed)
 - example input (if needed also word lists / hash lists)
 - further details when you first saw this problem (e.g. which hashcat version was the first to have this bug)
 - tell us if it can be reliably reproduce or it is kind of random
 - if possible test with different operating systems / setups (or at least mention the details of your system)
- New feature/algorithm:
 - name of the algorithm/feature
 - mention where the algorithm is currently being used (software, web app etc) or why the feature should be added
 - source code or source code link
 - mention any restrictions/limits or default lengths (maximum salt length, password length, ...)
 - several full example hash/plain pairs in case of a hashing algorithm

Where can I download the source code?

hashcat and hashcat legacy are released under the MIT license. You can download the source code from:

- hashcat legacy: <https://github.com/hashcat/hashcat-legacy>
- hashcat: <https://github.com/hashcat/hashcat>

You can also help the developers to improve hashcat/hashcat-legacy/hashcat-utils etc with problem reports and feature requests by opening github "issues".

What is Team Hashcat?

Team Hashcat is a team of hand selected enthusiasts who devoted themselves to represent the name Hashcat in cracking contests.

The team has participated in every major cracking competition in recent years, including Korelogic's "Crack Me If You Can" at Defcon (in 2017 CMIYC was held during Derbycon instead), "Hashrunner" at the Positive Hack Days and CracktheCon at Cyphercon (Milwaukee).